

CSE 214: DESIGN PATTERNS ONLINE

Creational Pattern Evaluation (Abstract Factory / Theme Framework)

Time Allowed: 20 minutes | Full Marks Solution

Problem Architecture Analysis

The task mandates a creational system managing a family of related user-interface components (Button, TextField, and Dialog) adhering to distinct stylistic themes (Light and Dark). To eliminate the risk of component mixing (e.g., matching a Light Button with a Dark Dialog) while letting the client decouple from concrete instantiation details, the **Abstract Factory Pattern** (implemented via a polymorphic Theme hierarchy) is utilized.

Complete Single-File Source Code

Save the following implementation entirely inside a single file named `Main.java`. The file contains all abstract interfaces, concrete variations, the factory layout, and a functional verification suite inside the entry point.

```
import java.util.*;

// =====
// 1. ABSTRACT PRODUCT INTERFACES
// =====

interface Button {
    void render();
}

interface TextField {
    void display();
}

interface Dialog {
    void show();
}

// =====
// 2. CONCRETE PRODUCTS: LIGHT THEME COMPONENT FAMILY
// =====

class LightButton implements Button {
    @Override
    public void render() {
        System.out.println("Rendering Light Button");
    }
}

class LightTextField implements TextField {
```

```

        @Override
        public void display() {
            System.out.println("Displaying Light Text Field");
        }
    }

    class LightDialog implements Dialog {
        @Override
        public void show() {
            System.out.println("Showing Light Dialog Box");
        }
    }

// =====
// 3. CONCRETE PRODUCTS: DARK THEME COMPONENT FAMILY
// =====

    class DarkButton implements Button {
        @Override
        public void render() {
            System.out.println("Rendering Dark Button");
        }
    }

    class DarkTextField implements TextField {
        @Override
        public void display() {
            System.out.println("Displaying Dark Text Field");
        }
    }

    class DarkDialog implements Dialog {
        @Override
        public void show() {
            System.out.println("Showing Dark Dialog Box");
        }
    }

// =====
// 4. ABSTRACT FACTORY INTERFACE (Theme Interface)
// =====

    interface Theme {
        Button createButton();
        TextField createTextField();
        Dialog createDialog();
    }

// =====
// 5. CONCRETE FACTORIES (Theme Implementations)
// =====

    class LightTheme implements Theme {
        @Override

```

```

    public Button createButton() {
        return new LightButton();
    }

    @Override
    public TextField createTextField() {
        return new LightTextField();
    }

    @Override
    public Dialog createDialog() {
        return new LightDialog();
    }
}

class DarkTheme implements Theme {
    @Override
    public Button createButton() {
        return new DarkButton();
    }

    @Override
    public TextField createTextField() {
        return new DarkTextField();
    }

    @Override
    public Dialog createDialog() {
        return new DarkDialog();
    }
}

// =====
// 6. CLIENT APPLICATION WORKER
// =====
class Application {
    private Button button;
    private TextField textField;
    private Dialog dialog;

    // The client relies purely on the Theme interface to avoid component mismatch
    public Application(Theme theme) {
        this.button = theme.createButton();
        this.textField = theme.createTextField();
        this.dialog = theme.createDialog();
    }

    public void renderUI() {
        button.render();
        textField.display();
        dialog.show();
    }
}

```

```

    }
}

// =====
// 7. ENTRY POINT MAIN CLASS
// =====
public class Main {
    public static void main(String[] args) {
        // Scenario 1: Executing system configuration via Light Theme
        System.out.println("--- Client Selecting Light Theme ---");
        Theme lightTheme = new LightTheme();
        Application lightApp = new Application(lightTheme);
        lightApp.renderUI();

        System.out.println();

        // Scenario 2: Dynamic swap out to Dark Theme without affecting core
        // execution logic
        System.out.println("--- Client Swapping to Dark Theme ---");
        Theme darkTheme = new DarkTheme();
        Application darkApp = new Application(darkTheme);
        darkApp.renderUI();
    }
}

```